

BrokerVerse

Out-of-the-box insurance broking platform

Solution Architecture · Document 01 of 11

Solution Component Diagram and Application Architecture

Product	BrokerVerse
Document	01 Solution Component Diagram and Application Architecture
Version	1.0
Date	29 September 2026
Prepared by	iorta TechNXT
Classification	Confidential: for the client, iorta TechNXT and authorised partners

Document control

Document	BrokerVerse Solution Architecture: 01 Solution Component Diagram and Application Architecture
Document ID	BV-SA-01
Version	1.0
Status	Issued for review
Date	29 September 2026
Prepared by	iorta TechNXT
Source baseline	BrokerVerse source repository, branch brokerverse-platform (backend/, brokerverse/, docker-compose.yml, render.yaml, docs/)

Version history

Version	Date	Author	Change
1.0	29 September 2026	iorta TechNXT	Initial issue

Review and approval

Role	Name	Signature	Date
Solution architect			
DevOps / infrastructure lead			
Information security			
Business owner			

Conventions

- Statements of fact are derived from the code, configuration and database of the baseline above (file paths are given relative to the repository). Figures measured on the local test system are labelled **test data** or **indicative**.
- Targets, sizes, retention periods and thresholds that are not facts of the code are labelled **Recommended / to be confirmed by the business and DevOps**.
- Items marked **Gap** are defects or missing capabilities found while preparing this document.

Contents

1 Introduction.....	4
1.1 Purpose and scope.....	4
1.2 Architecture at a glance.....	4
2 Solution component diagram.....	5
2.1 Logical components.....	5
2.2 Front-end application structure.....	6
3 Deployment view.....	7
3.1 Target deployment on AWS.....	7
3.2 Alternative deployments.....	9
4 Request processing.....	11
4.1 API request pipeline.....	11
4.2 Authentication and session flow.....	13
5 Module map.....	15
5.1 Backend modules.....	15
5.2 Front-end areas and backend modules.....	17
6 Integration points.....	18
7 Security architecture summary.....	19
7.1 Roles and permissions.....	19
7.2 Controls.....	19
8 Key business flows.....	21
8.1 Lead, quotation, policy and billing.....	21
8.2 Receipt, commission and remittance.....	22
8.3 Direct bill.....	23
8.4 Claims.....	24
8.5 Renewal.....	25
8.6 Scheduled jobs.....	25
9 Architecture decisions and constraints.....	27
10 Related documents.....	28
11 Glossary.....	29

1 Introduction

1.1 Purpose and scope

This document describes the application architecture of BrokerVerse, the out-of-the-box insurance broking platform of iorta TechNXT: its logical components, how it is deployed, how a request is processed and secured, how the functional modules map to the front end and the backend, which external systems it integrates with, and how the main business flows run through the components. It is the entry point to the Solution Architecture set; the database, technology, shared services and operational documents (02 to 11) go into detail.

The description is derived from the code and configuration of the baseline named in the document control section: the React front end in `brokerverse/`, the Node.js API in `backend/`, `docker-compose.yml`, `render.yaml`, `docs/DEPLOY.md` and `docs/GO_LIVE_CHECKLIST.md`.

1.2 Architecture at a glance

Aspect	BrokerVerse
Style	Three-tier web application: single-page application (SPA), stateless REST API, relational database; modular monolith on the server
Presentation tier	React 18 SPA built with Create React App / craco, PrimeReact components, Redux Toolkit, served as static files (S3 + CloudFront in the target, nginx in the container option)
Application tier	Node.js 22 (ESM) with Express 4; 32 module routers mounted under <code>/api</code> , 568 registered routes plus two health endpoints; built-in cron scheduler
Data tier	PostgreSQL 16: 102 tables, 32 migration files applied on start; files on a persistent volume (<code>UPLOAD_DIR</code>)
Integration	SMTP (outbound e-mail through an outbox table); public quote-approval link for clients; OpenAPI / Postman / Excel API documentation generated from the route registry
Security	JWT access tokens (30 minutes) with rotating refresh tokens (30 days), token version revocation, role and permission checks on every route, record scope for agents, maker-checker approvals, audit log, TOTP two-factor, HMAC-signed file links
Deployment target	AWS ap-southeast-1: existing front-end pipeline (GitHub Actions, S3, CloudFront) and a new backend container with PostgreSQL (e.g. Amazon RDS) and a persistent upload volume (e.g. Amazon EFS). Alternatives: Docker Compose and a Render blueprint

2 Solution component diagram

2.1 Logical components

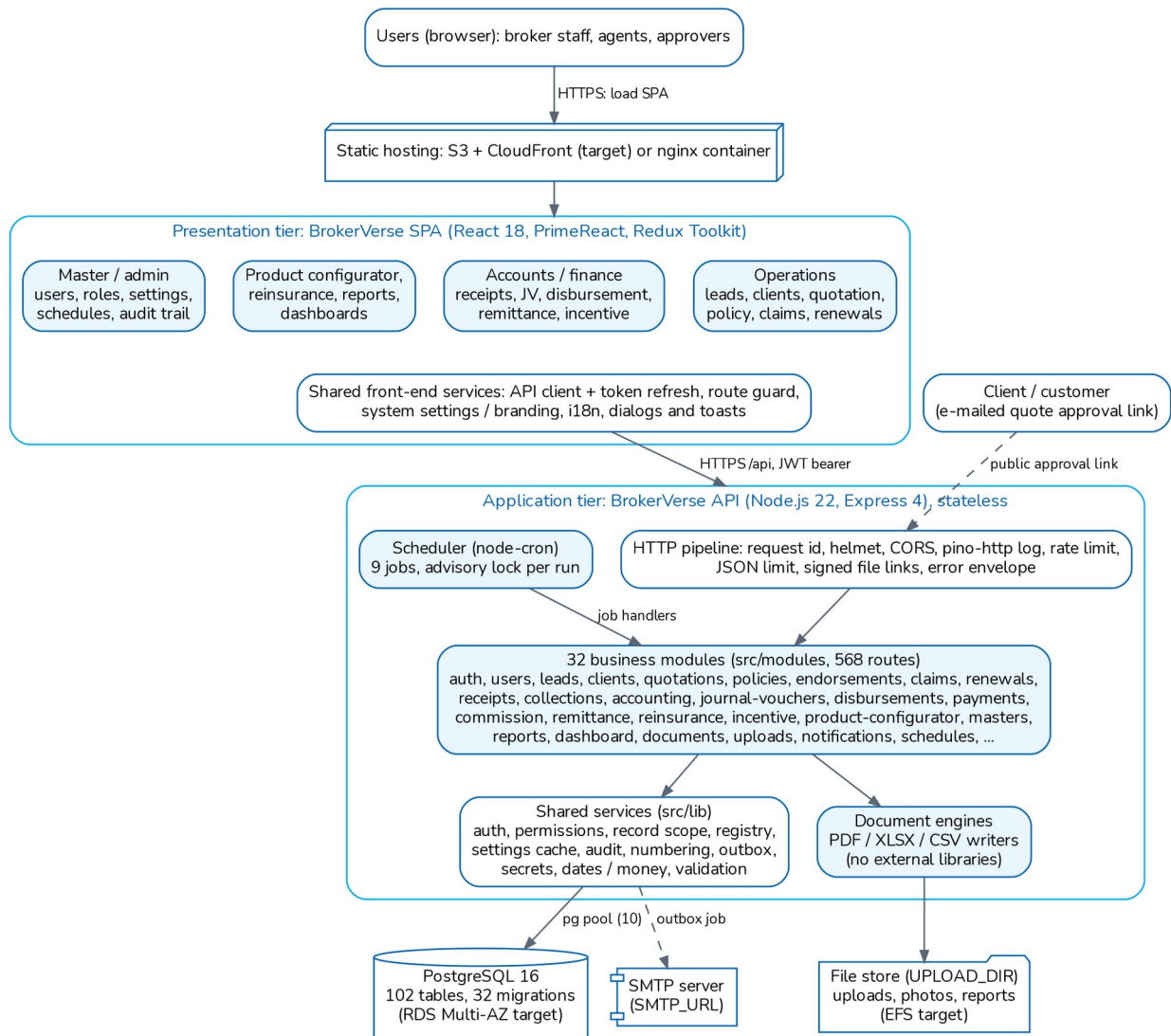


Figure 1. Logical component view of BrokerVerse

Component	Responsibility
Browser SPA	All user screens: operations (leads to renewals), accounts and finance, commission, product configurator, reinsurance, reports, dashboards, master data and administration. Holds the session tokens, renews them automatically and hides menus and routes the user's roles may not open.
Static hosting	Serves the compiled SPA (HTML, JavaScript, CSS, images). No server-side rendering; every data call goes to /api.
HTTP pipeline	Cross-cutting request handling in <code>backend/src/app.js</code> : correlation id, security headers (helmet), CORS, structured access log with redaction, global rate limit, JSON size limit, signing of file URLs in responses, uniform error envelope.
Business modules	32 folders in <code>backend/src/modules</code> , each with a <code>router.js</code> (endpoints declared through the route registry) and services holding the business rules and SQL. Loaded automatically at start (<code>loadModules()</code>).

Component	Responsibility
Shared services	<code>backend/src/lib</code> : authentication, permissions, record scope, settings cache, audit, e-mail outbox, secrets and signed links, dates in the business time zone, money rounding, validation, rate limits, health (document 05).
Scheduler	<code>backend/src/jobs/scheduler.js</code> : node-cron schedules the enabled rows of <code>scheduled_jobs</code> ; each run takes a PostgreSQL advisory lock so it runs once across instances and is recorded in <code>job_runs</code> .
Document engines	Dependency-free writers in <code>backend/src/tools</code> and <code>backend/src/modules/documents</code> : PDF (quotations, policy schedules, receipts, letters, report files), XLSX (reports, exports) and CSV.
PostgreSQL	System of record for all business, financial, configuration, security and log data (document 02 and 03).
File store	<code>UPLOAD_DIR</code> (default <code>./uploads</code> , <code>/app/uploads</code> in the container): uploaded documents and photos, generated reports and statements; registered in the <code>documents</code> table.
SMTP server	Delivers e-mail queued in <code>email_outbox</code> (quotation approval links, password reset codes, renewal and collection reminders, report schedules, remittance statements).

2.2 Front-end application structure

The SPA (`brokerverse/src`, about 880 JavaScript / JSX files) is organised by functional area rather than by technical layer:

Folder	Content
<code>agentModule/</code>	Operations screens: leads, quotation wizard, policy, endorsements, claims, renewals, collections, open items, payments, agent dashboard, customer quote approval page
<code>module/</code>	Back-office areas: Administration, ClaimsModule, Commission, CorrectionJV, ExecutiveDashboard, FinanceMastersModule, GeneralMasters, Incentive, JournalVoucher, PaymentVoucher, PettyCashManagement, ProductConfigurator, Receipts, Reinsurance, Remittance, Renewal, Reports, Reversals, SystemSettings, UnderwritingModule, accounts
<code>services/</code>	36 API client services (one per backend area, e.g. <code>policyService.js</code> , <code>remittanceService.js</code>)
<code>routes/</code>	<code>MainRoute.js</code> (about 400 routes), <code>ProtectedRoute</code> (route guard), <code>apiRoutes.js</code>
<code>redux/, context/, hooks/</code>	Redux Toolkit store (main and agent reducers, reset on sign-out), notification context, currency formatting hook
<code>utility/, utils/</code>	Shared helpers: HTTP interceptor and session refresh, menu permissions, dialogs and toasts, date / currency / phone formatting, idle time-out, system settings application
<code>components/</code>	Reusable UI: layout, header, side bar, file upload, dialogs, toasts, status badges, stepper, notification drop-down
<code>locales/, i18n.js</code>	i18next resources: English (<code>en.json</code>) and Thai (<code>th.json</code>); Filipino is configured in settings but has no translation file

3 Deployment view

3.1 Target deployment on AWS

The go-live checklist ([docs/GO_LIVE_CHECKLIST.md](#)) keeps the existing front-end pipeline and adds a backend service. Figure 2 shows the recommended target; items that are not yet provisioned (WAF, ECR image pipeline, CloudWatch alarms) are recommendations.

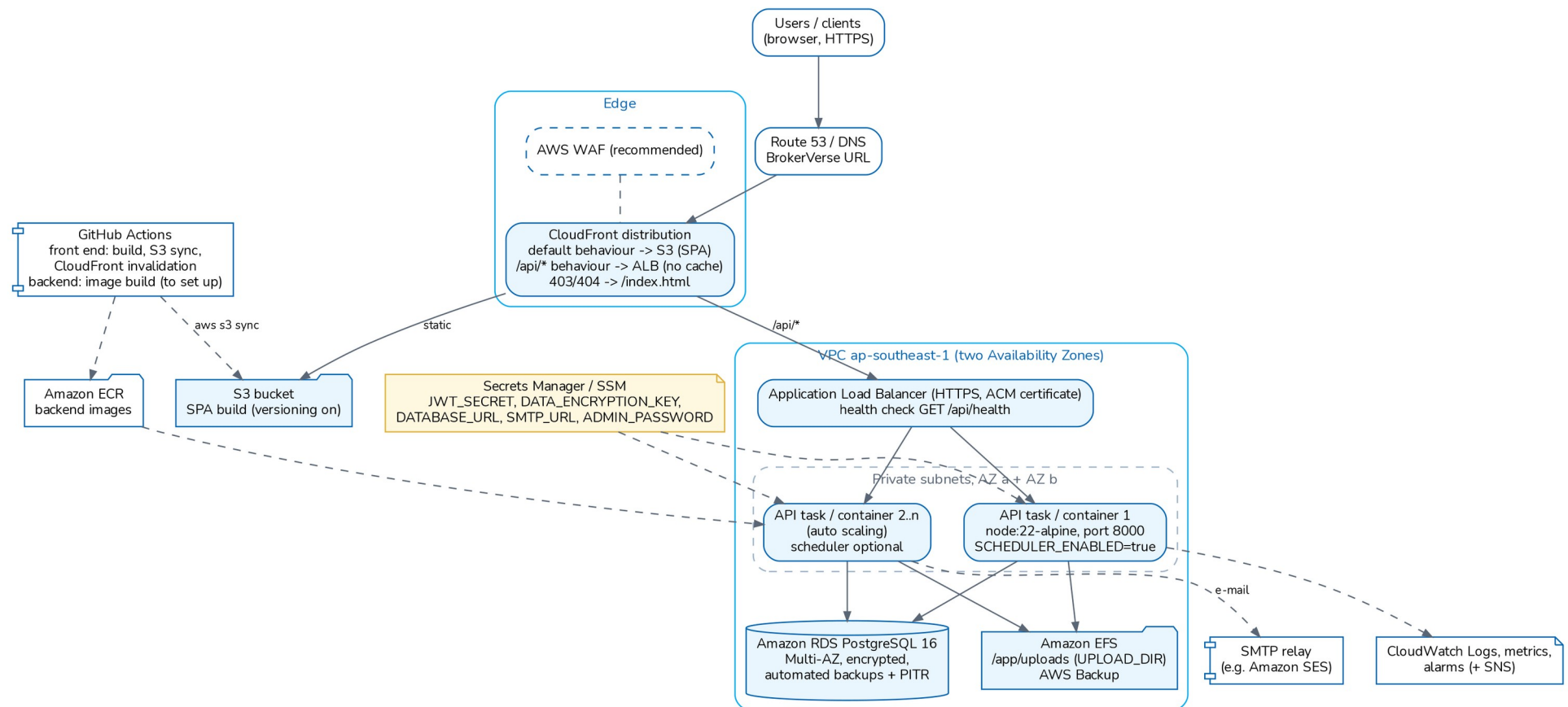


Figure 2. Target deployment on AWS ap-southeast-1 (recommended; to be confirmed by DevOps)

Layer	Fact from the baseline	Recommended / to be confirmed by the business and DevOps
Front-end build and hosting	<code>brokerverse/.github/workflows/deploy.yml</code> : on push to dev, Node 20, <code>npm install</code> , <code>npm run build</code> , <code>aws s3 sync build/ s3://\$S3_BUCKET --delete</code> , CloudFront invalidation <code>/*</code> , region <code>ap-southeast-1</code>	Add <code>REACT_APP_BASE_URL</code> to the build step (required, see checklist section 3); use GitHub OIDC role instead of long-lived access keys; enable S3 versioning; re-enable the commented-out test job
API address	Same-domain option recommended: CloudFront behaviour <code>/api/*</code> to the backend load balancer, all methods and headers, no caching	Custom error responses 403/404 to <code>/index.html</code> (SPA deep links)
API runtime	<code>backend/Dockerfile</code> : <code>node:22-alpine</code> , <code>npm ci --omit=dev</code> , runs as user <code>node</code> , port 8000, <code>HEALTHCHECK</code> on <code>/api/health</code>	ECS Fargate service (or App Runner / EC2) in private subnets of two AZs, minimum two tasks behind an ALB
Database	PostgreSQL 16, created empty; the API migrates and seeds it on start	Amazon RDS for PostgreSQL 16, Multi-AZ, encryption at rest, <code>sslmode=require</code> , automated backups with PITR
Files	Persistent volume at <code>UPLOAD_DIR</code> required (a container without it loses documents on restart)	Amazon EFS mounted at <code>/app/uploads</code> in every task, backed up by AWS Backup
Secrets	<code>JWT_SECRET</code> , <code>DATA_ENCRYPTION_KEY</code> , <code>ADMIN_PASSWORD</code> , <code>DATABASE_URL</code> , <code>SMTP_URL</code> from the environment; production start refuses unsafe values	AWS Secrets Manager / SSM Parameter Store injected into the task definition

3.2 Alternative deployments

Docker Compose (`docker-compose.yml`, own server or VM): three services. `db` runs `postgres:16-alpine` with a `pg_isready` health check and the volume `db-data`; `api` is built from `backend/`, runs with `NODE_ENV=production` by default, refuses to start without `JWT_SECRET`, `DATA_ENCRYPTION_KEY`, `ADMIN_PASSWORD`, `CORS_ORIGINS` and `PUBLIC_BASE_URL`, and keeps files in the volume `uploads`; `web` is built from `brokerverse/` (nginx 1.27) and publishes port 8080, serving the SPA and forwarding `/api/` to `http://api:8000/api/`. A TLS proxy must be placed in front of port 8080.

Render blueprint (`render.yaml`): a managed PostgreSQL (`basic-256mb` plan), the API as a Docker web service with `healthCheckPath: /api/health`, generated `JWT_SECRET` and `DATA_ENCRYPTION_KEY` and a 1 GB disk at `/app/uploads`, and the SPA as a static site with a rewrite of `/*` to `/index.html`. Suitable for demonstrations and UAT, not sized for production.

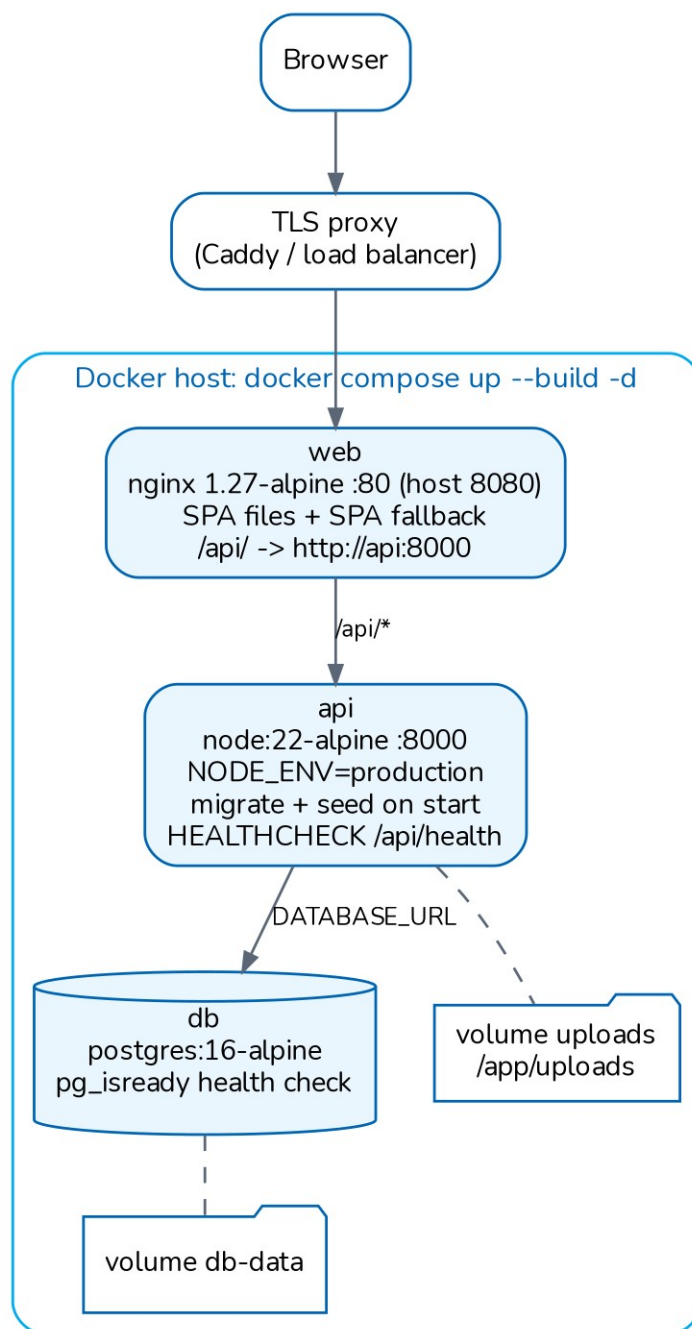


Figure 3. Docker Compose deployment (docker-compose.yml)

4 Request processing

4.1 API request pipeline

Every request passes the same middleware chain before it reaches a handler. The route registry (`backend/src/lib/registry.js`) places `requireAuth` first on every route not declared `auth: false`, so an endpoint cannot be left open by omission; 14 routes are public by design (sign-in, refresh, logout, password reset and policy, public system settings, version, customer quote approval, file download with a signed link, generated report download with a signed token).

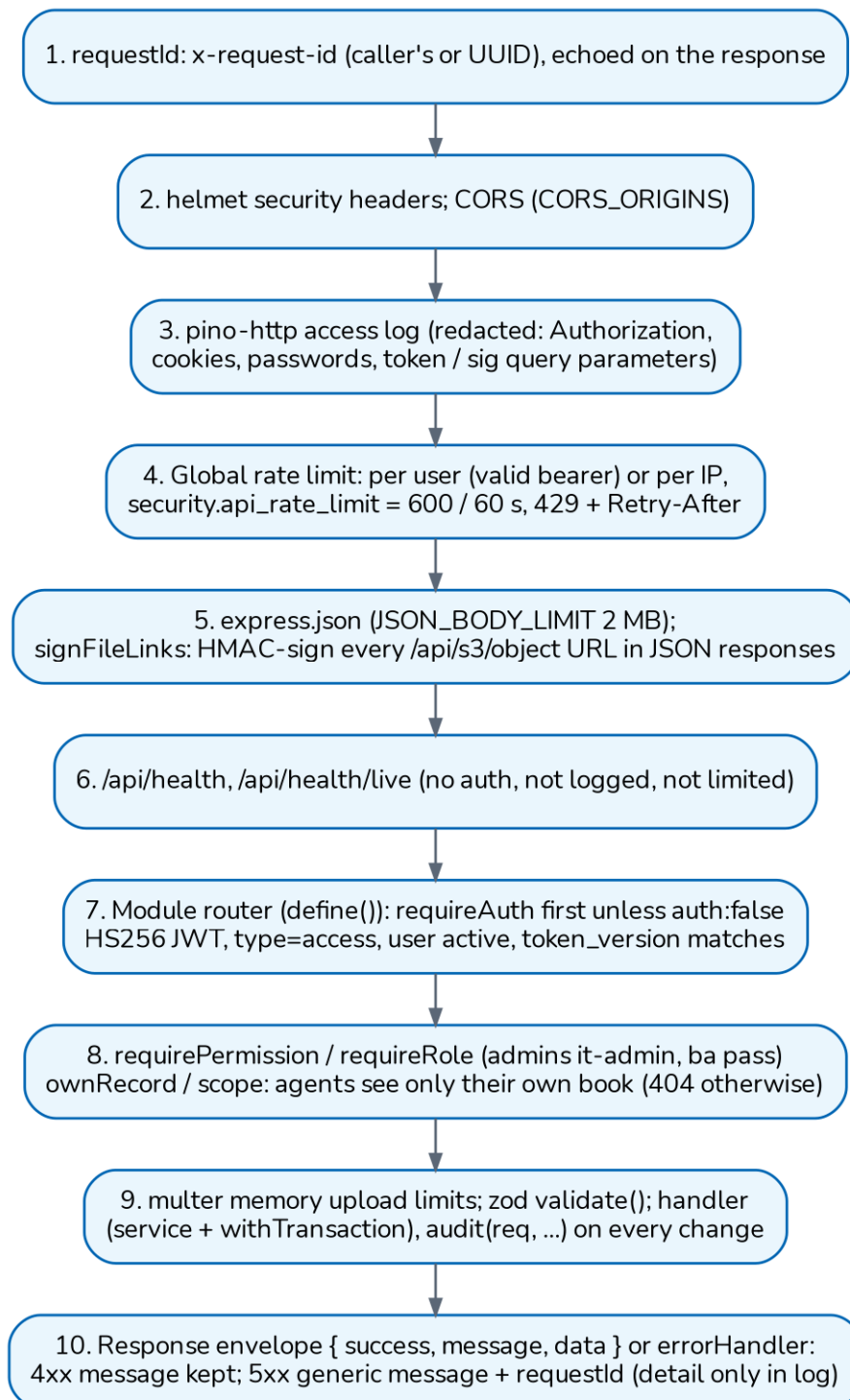


Figure 4. Request pipeline of the BrokerVerse API

4.2 Authentication and session flow

Sign-in, token refresh and signed file links

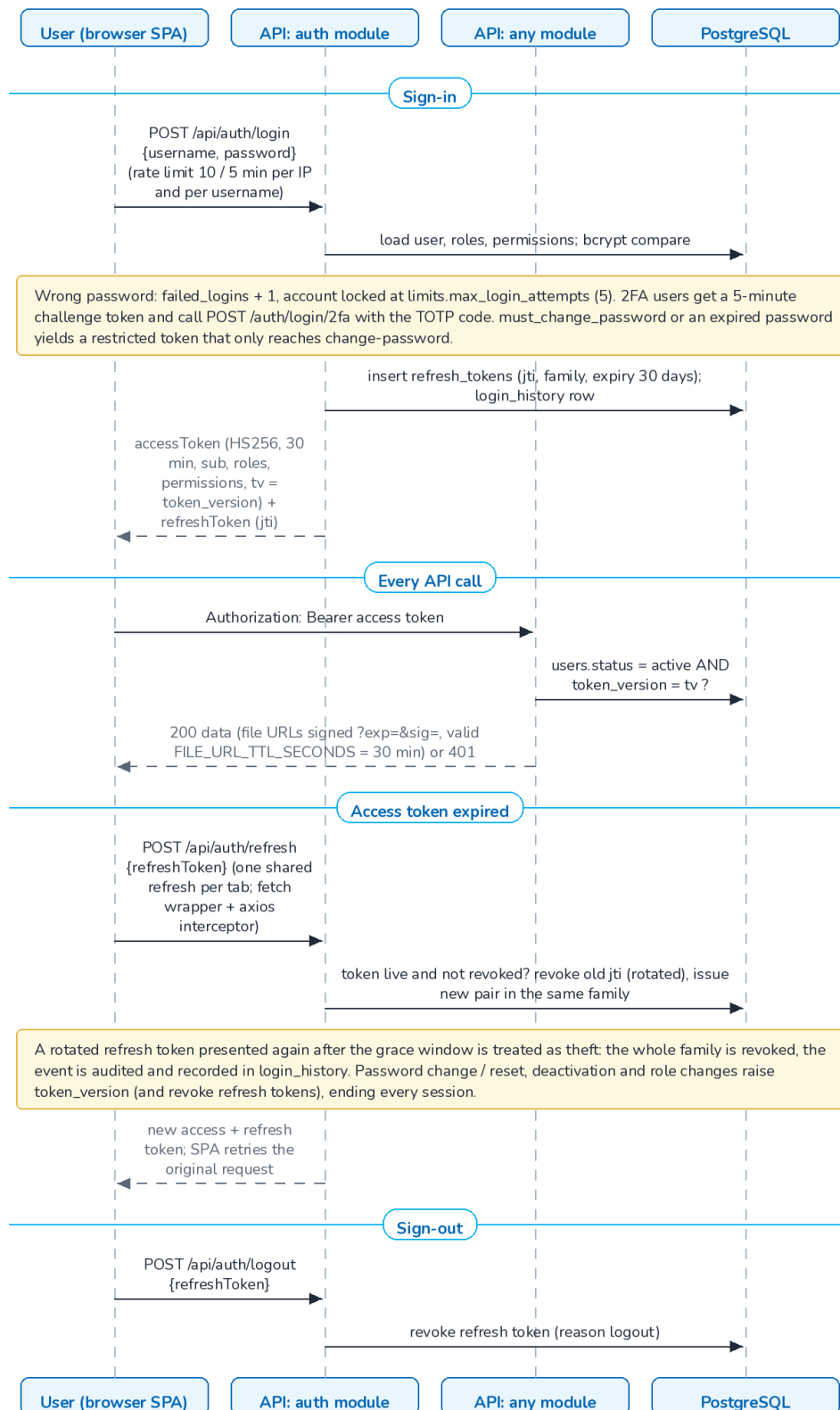


Figure 5. Sign-in, refresh-token rotation and signed file links

Mechanism	Implementation
Access token	JWT, HS256 only, lifetime <code>JWT_ACCESS_TTL_SECONDS</code> (default 1,800 s = 30 minutes); claims: user id, username, roles, permissions, <code>type=access</code> , <code>tv</code> (token version) and an optional <code>restrict</code> (<code>enrol2fa</code> , <code>pwchange</code>)
Per-request check	<code>authenticate()</code> verifies the signature and type, then reads <code>users.status</code> and <code>users.token_version</code> ; an inactive user or a changed token version ends the session at once (HTTP 401)
Refresh token	JWT with a <code>jti</code> recorded in <code>refresh_tokens</code> (lifetime <code>JWT_REFRESH_TTL_SECONDS</code> , default 30 days); rotated on every use; reuse of a rotated token after the grace window revokes the whole token family and is audited
Revocation	Password change or reset, administrator reset, deactivation and role / permission changes raise <code>token_version</code> and revoke refresh tokens (<code>revokeSessions()</code>)
Sign-in protection	In-memory limit of 10 attempts per 5 minutes per IP and per username (<code>security.login_rate_limit</code>); account locked after <code>limits.max_login_attempts</code> (5) wrong passwords; every attempt recorded in <code>login_history</code>
Password policy	Minimum length 8, upper / lower case, digit and symbol, history of 5, maximum age 90 days (<code>security.password_*</code>); must-change and expired passwords get a restricted token
Two-factor	TOTP (RFC 6238) in <code>lib/totp.js</code> ; secrets encrypted at rest with AES-256-GCM under <code>DATA_ENCRYPTION_KEY</code> ; roles that must use it are configured in <code>security.require_2fa_roles</code> (empty by default)
Front end	Tokens kept in <code>localStorage</code> ; <code>utility/sessionRefresh.js</code> wraps <code>fetch</code> and <code>utility/interceptor.js</code> wraps <code>axios</code> with one shared refresh; idle sign-out after <code>limits.session_idle_minutes</code> (30) with a one-minute warning
File links	Stored documents are served by <code>GET /api/s3/object/<key></code> only with a bearer token or an HMAC-signed link (<code>?exp=&sig=</code>) valid for <code>FILE_URL_TTL_SECONDS</code> (1,800 s); the API signs every object URL in JSON responses on the way out

Limitation: Tokens are stored in the browser's local storage (accepted for go-live in the checklist, section 9). A later change to an `httpOnly`, `SameSite` refresh cookie needs a backend change. The global and sign-in rate limits are counted per API process, so with several instances the effective limit is multiplied by the number of instances.

5 Module map

5.1 Backend modules

The API is a modular monolith: each module owns its routes, services and tables, and shares the database and the `lib/` services. Module routers are loaded in file-name order (the `system` module first) and mounted under `/api`; some modules mount extra prefixes (for example `clients` also serves `/customers`, `leads` serves `/lead` and `/leads`).

Module (backend/src/modules)	Routes	Responsibility
auth	15	Sign-in, two-factor challenge and enrolment, refresh, logout, change / forgot / reset password, profile
users	16	Users and roles (User Management), role permissions, sessions, sign-in history
leads	11	Leads / prospects, statistics, lead report
clients	6	Clients and customer codes (<code>/clients</code> , <code>/customers</code>)
addresses	5	Provinces, cities, districts and postal codes
quotations	38	Quotation wizard, motor tariff, premium calculation, customer approval link, submission to insurer, conversion to policy, quotation masters, e-mail
policies	12	Policy list and detail, KYC, payment capture for finance confirmation, policy documents
endorsements	8	Endorsements and cancellations with re-pricing and billing
documents	5	Server-rendered PDFs: quotation, policy schedule, receipt
claims	13	Claim registration with acceptance check, lifecycle, settlement with maker-checker, documents and letters, claims reports
renewals	42	Renewal pipeline, renewal quotes, notices, batches and queue, workspace, analytics, win-back campaigns
receipts	14	Official receipts, application to bills, billing statements, receivables
collections	9	Collection items, follow-up actions, due-date reminders, KPIs
accounting	24	Accounting entries, ledgers per client / policy, open-item matching, trial balance, period close
journal-vouchers	8	Journal vouchers with maker-checker, correction and reversal
disbursements	15	Payment vouchers, invoice lists, cheques, commission and insurer payouts
payments	26	Open items, payments and petty cash (funds, requests, disbursements, receipts, replenishment)
commission	19	Commission accrual, eligibility, approval, payout; referrer accounts and dashboard
remittance	85	Remittances to insurers, bills, settlements, multi-level approvals and delegations, statements, direct bill (debit notes, collections)
reinsurance	38	Reinsurers, treaties, cessions, recoveries, bordereaux, reconciliation, analytics
incentive	22	Incentive programmes, calculation batches with maker-checker, results, reports
product-configurator	72	Product templates, components, rating, rules, risk mapping, versions
masters	11	Generic master data screens driven by <code>master_types</code>
reports	13	Report catalogue, on-screen run, file generation (CSV / XLSX / PDF), schedules
dashboard, search	6	Executive, sales, underwriting, claims and agent dashboards; global search
notifications	8	In-app notifications, unread count
uploads	11	File upload, signed download, existence check, delete (<code>/s3</code> , <code>/upload</code>)

Module (backend/src/modules)	Routes	Responsibility
settings, system-settings	11	Configuration keys by group; branding and system settings (public read for the sign-in page)
schedules	4	Scheduled jobs: list, run history, edit, run now
system	1	GET /api/version: build and runtime information

Route counts are those of the route registry and `backend/docs/api/openapi.json` (568 operations: 271 GET, 210 POST, 57 PUT, 23 DELETE, 7 PATCH); renewals combines the `policy-renewals` and `renewals` prefixes, payments combines open items, payments and petty cash.

5.2 Front-end areas and backend modules

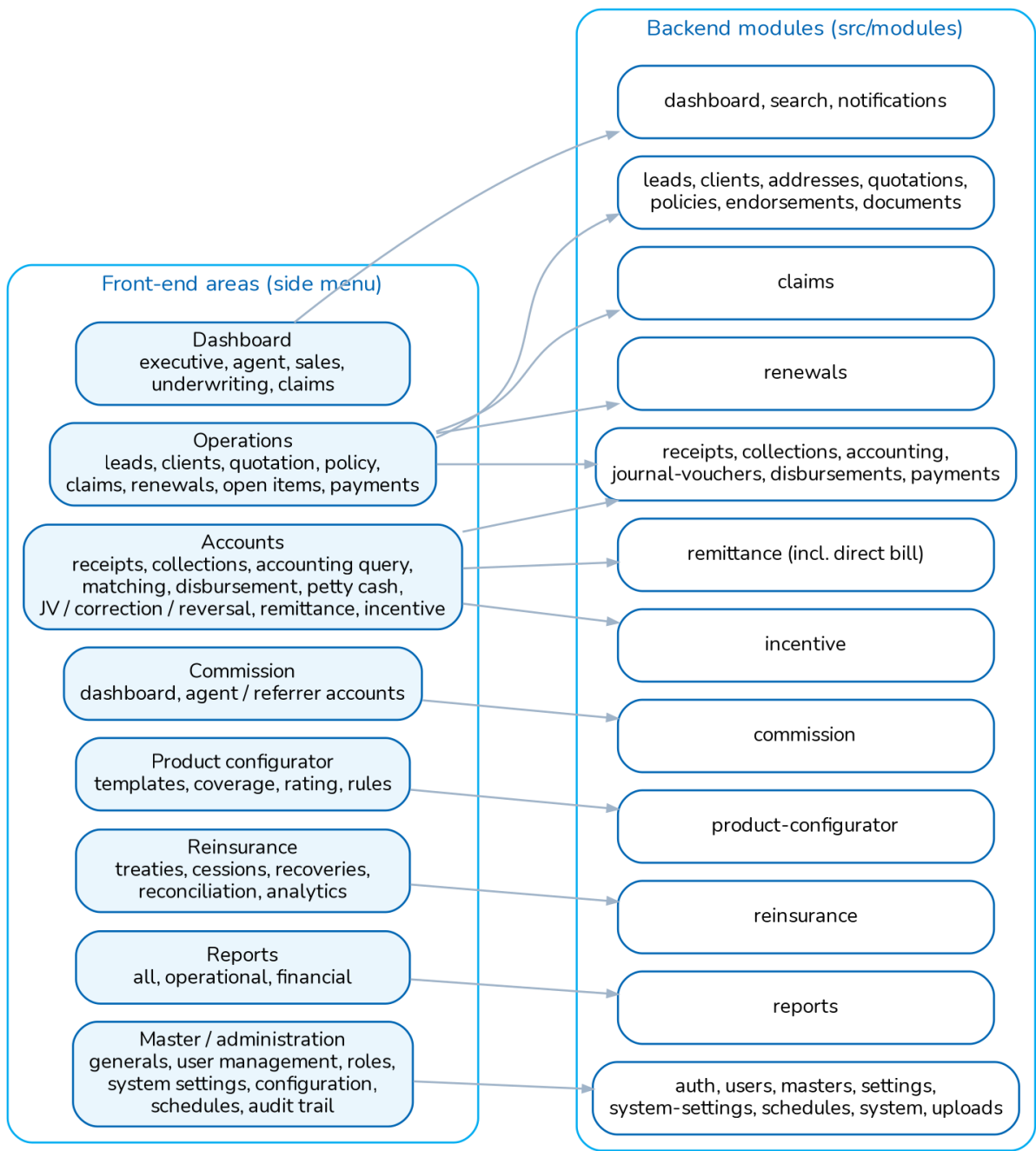


Figure 6. Front-end areas (menus) and the backend modules they use

6 Integration points

Integration	Direction	Description
SMTP e-mail	Outbound	<code>lib/mailer.js</code> queues messages in <code>email_outbox</code> ; the <code>email-outbox</code> job (every 5 minutes) sends up to 50 queued messages per run through <code>nodemailer</code> and <code>SMTP_URL</code> when <code>notification.email_enabled</code> is true; up to 5 attempts, then <code>failed</code> . Sender: <code>notification.from_address</code> .
Client quote approval	Inbound (public)	The approval e-mail carries a signed token (<code>quotations.approval_link_ttl_hours</code> , 168 h); <code>POST /api/quotations/approve-by-customer</code> previews or accepts; only the latest token of a quotation is valid.
Insurers	Outbound (e-mail, files)	Quotation submission to the insurer's underwriting contact, remittance statements and bills, commission debit notes; CSV / XLSX / PDF files. No insurer API integration in the baseline.
Reinsurers	Outbound (files)	Bordereaux and reconciliation files generated as CSV.
Spreadsheet imports	Inbound (files)	Bulk uploads (for example disbursements, remittance items, policies) through <code>importUpload()</code> : one file of at most <code>IMPORT_MAX_MB</code> (10 MB), decompressed size capped at <code>IMPORT_MAX_INFLATED_MB</code> (50 MB) and <code>IMPORT_MAX_ROWS</code> (20,000) rows.
Payment gateway	Not integrated	Policy payments are captured by staff with proof of payment (<code>policy_payments</code>) and confirmed by finance; the payment screen shows a gateway only if configured.
API documentation	Developer	<code>npm run export:api</code> generates <code>backend/docs/api/openapi.json</code> , a Postman collection and environment, and <code>BrokerVerse_API_Touchpoints.xlsx / .csv</code> from the route registry.
Monitoring	Operations	<code>GET /api/health</code> (readiness), <code>GET /api/health/live</code> (liveness), <code>GET /api/version</code> (build, commit from <code>GIT_COMMIT</code> , database reachability, pending migrations); JSON logs on stdout (document 11).

7 Security architecture summary

7.1 Roles and permissions

Permissions are codes `read:<module>` and `write:<module>` for 25 modules (50 permission rows). Nine roles are seeded (`backend/src/db/seed.js`); administrators can create further roles and change grants in User Management.

Role	Access (seeded grants)
it-admin, ba	Administrator roles: every permission (checked in code as administrator)
user-access-admin	Users, roles, audit trail (read), notifications, settings (read); cannot grant administrator roles or change its own access
sales	Leads, clients, quotations, policies, endorsements, renewals, reports, notifications; read products, masters, claims
underwriting	Clients, quotations, policies, endorsements, renewals, reinsurance, products, reports; read leads, masters, claims
customer-services	Leads, clients, quotations, policies, endorsements, renewals, reports; read claims, masters, products
claims	Claims, reports; read clients, policies, masters
finance	Receipts, collections, disbursements, commission, remittance, incentive, journal vouchers, reports; read clients, policies, claims, masters, schedules
agent	Own leads, quotations, policies, endorsements and claims (first notice of loss); read clients; record-scoped

The server enforces permissions on every route (`requirePermission`, `requireRole`); the front end additionally hides menus and blocks routes by role (`utils/menuPermissions.js`, deny by default, and `routes/ProtectedRoute`). Receipts are finance-only (segregation of duties).

7.2 Controls

Control	Implementation
Record scope	Users whose roles are all listed in <code>security.scoped_roles</code> (default agent) see and act only on records they own or that belong to their clients (<code>lib/scope.js</code>); someone else's record answers 404 so its existence is not disclosed.
Maker-checker	A different user must approve: journal vouchers, payment vouchers and cheques, commission payouts, remittances and settlements (multi-level with delegation), commission debit notes, claim settlements, incentive calculations, reinsurance treaties, petty cash requests, quotation and renewal approvals.
Audit	<code>lib/audit.js</code> writes <code>audit_log</code> (user, entity, id, action, before / after JSON, IP) from every mutating handler; claims keep a field-level trail (<code>claim_field_changes</code>), quotations expose an audit trail; sign-ins in <code>login_history</code> .
Input validation	zod schemas on request bodies and parameters where declared; parameterised SQL only (no request values concatenated into SQL; dynamic identifiers white-listed).
Uploads	Type checked against <code>uploads.allowed_types</code> by file signature, detected type stored; per-file size <code>UPLOAD_MAX_MB</code> (10 MB) and <code>UPLOAD_MAX_FILES</code> (10); files served with <code>nosniff</code> and a sandbox CSP, non-image / non-PDF files as downloads.
Transport and headers	TLS at the load balancer / CloudFront; helmet headers on the API; nginx adds <code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> , <code>Referrer-Policy</code> ; CORS limited to <code>CORS_ORIGINS</code> (* refused in production).

Control	Implementation
Secrets	Production start refuses a missing, placeholder or short (< 32 characters) <code>JWT_SECRET</code> / <code>DATA_ENCRYPTION_KEY</code> , equal keys, <code>CORS_ORIGINS=*</code> or a localhost <code>PUBLIC_BASE_URL</code> . Reset codes are stored only as HMAC hashes.
Logging	pino JSON logs with redaction of authorization headers, cookies, passwords, refresh tokens and token / signature query parameters; 5xx responses return a generic message with the request id.

8 Key business flows

8.1 Lead, quotation, policy and billing

Lead to quotation to policy to billing

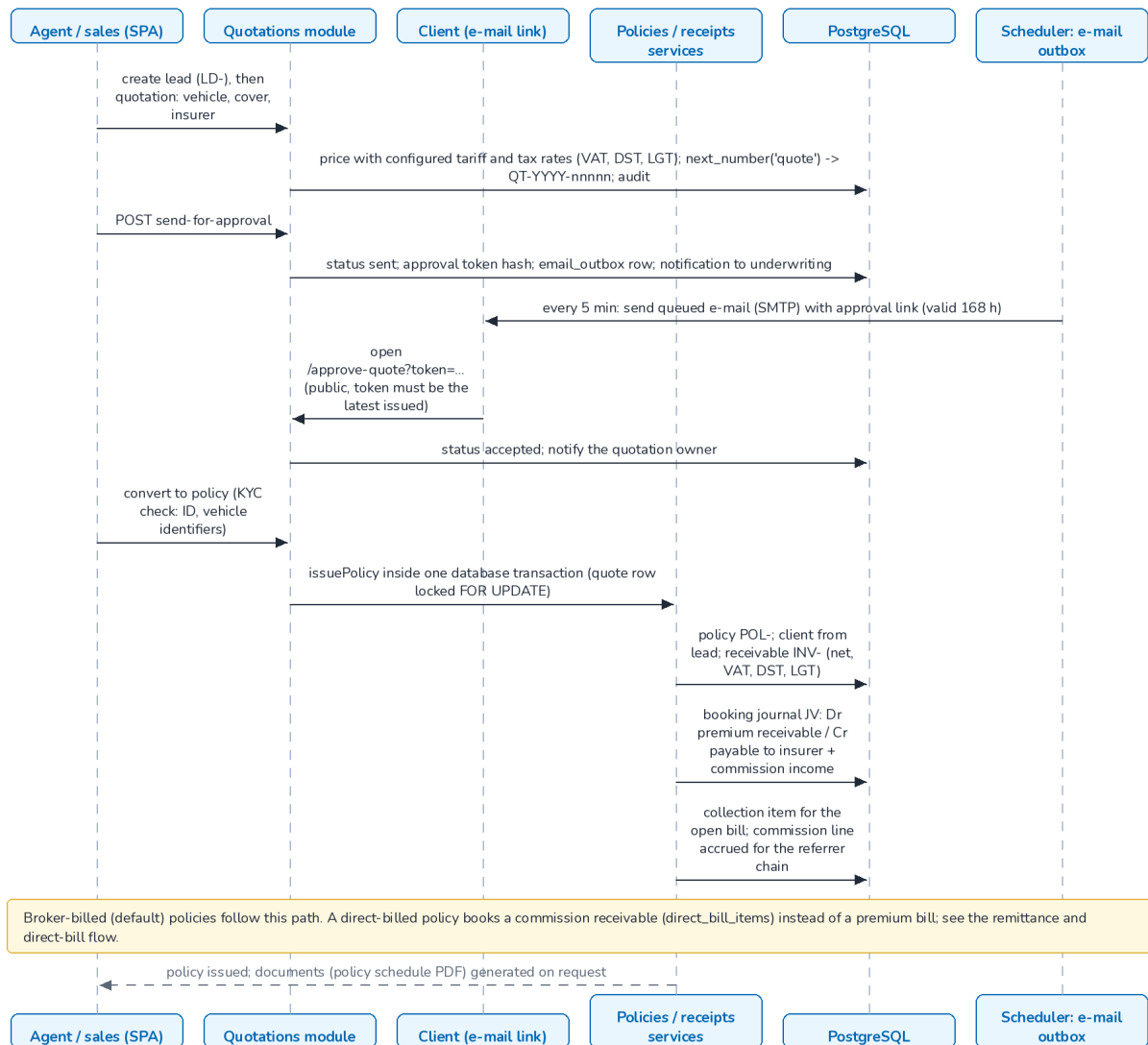


Figure 7. Lead to quotation to policy to billing

8.2 Receipt, commission and remittance

Receipt, commission payout and remittance to the insurer

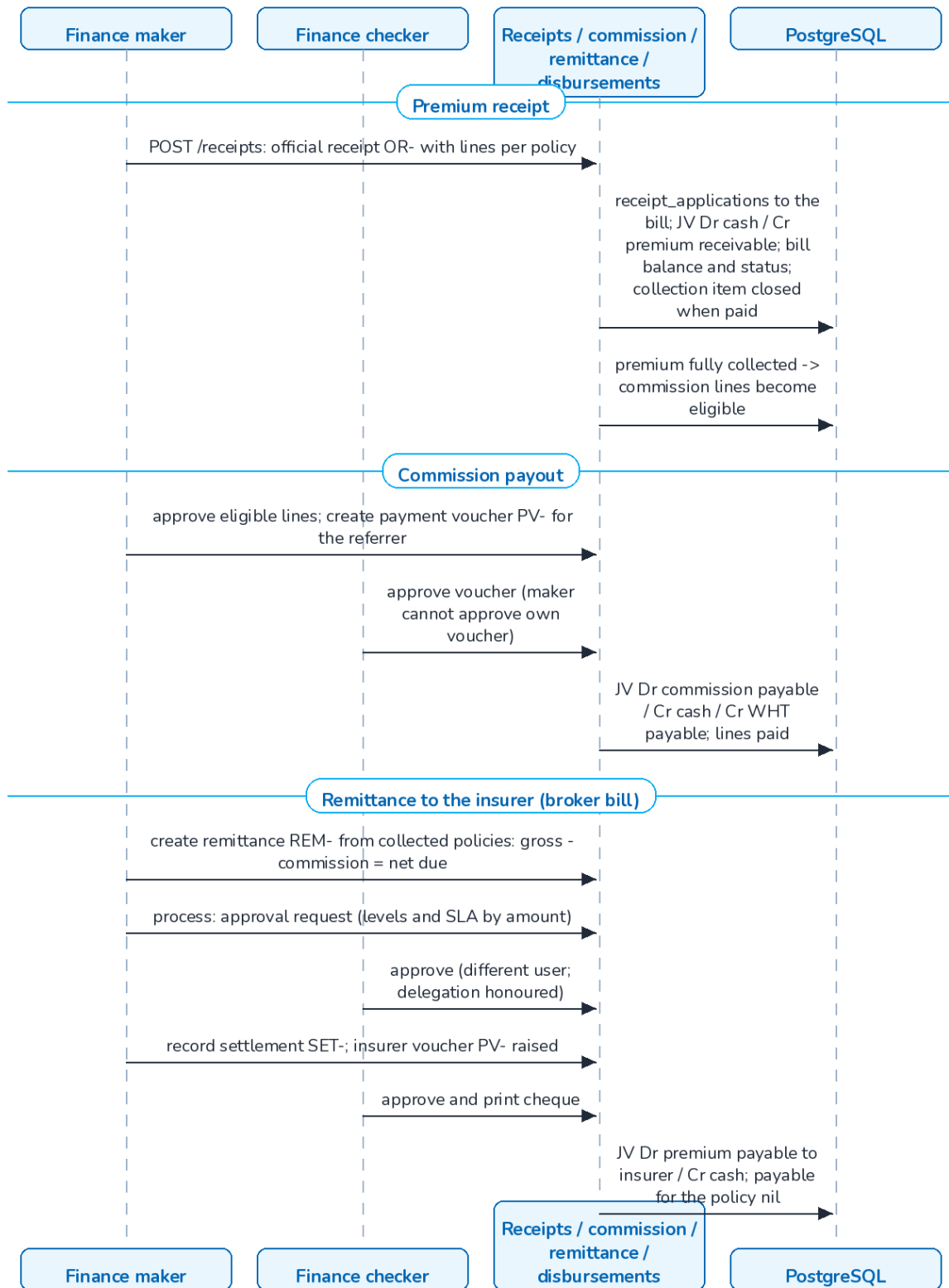


Figure 8. Receipt, commission payout and remittance to the insurer

8.3 Direct bill

Direct bill: commission debit note to the insurer

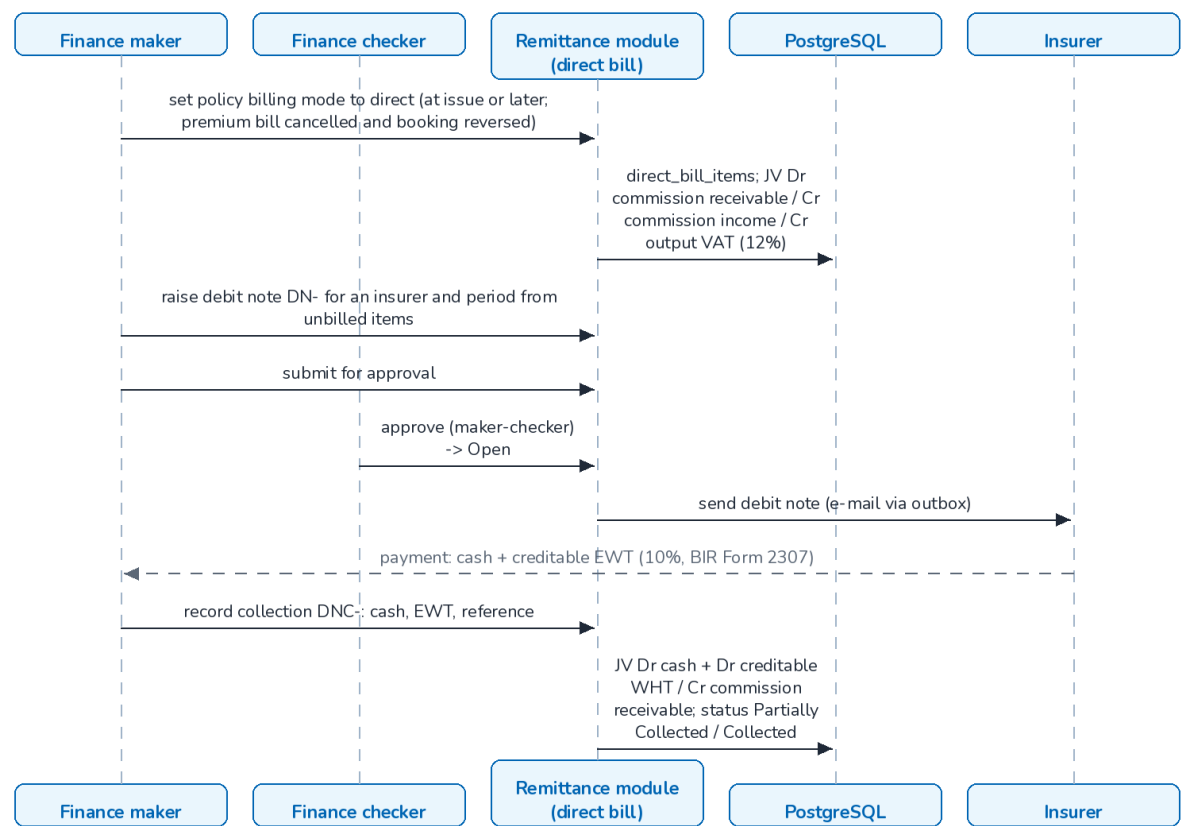


Figure 9. Direct bill: commission debit note to the insurer

8.4 Claims

Claim registration to settlement

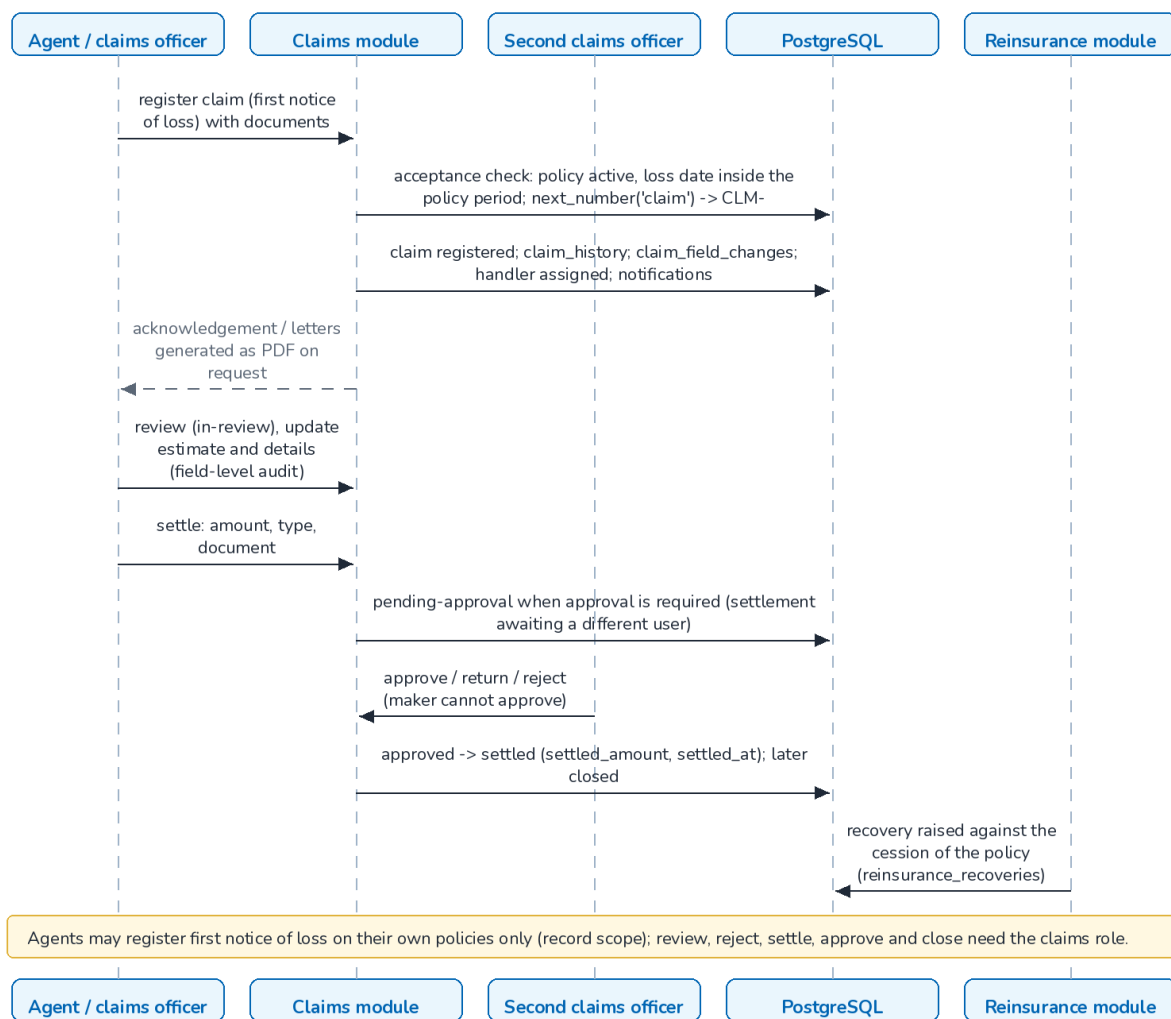


Figure 10. Claim registration to settlement

8.5 Renewal

Renewal pipeline to renewed policy

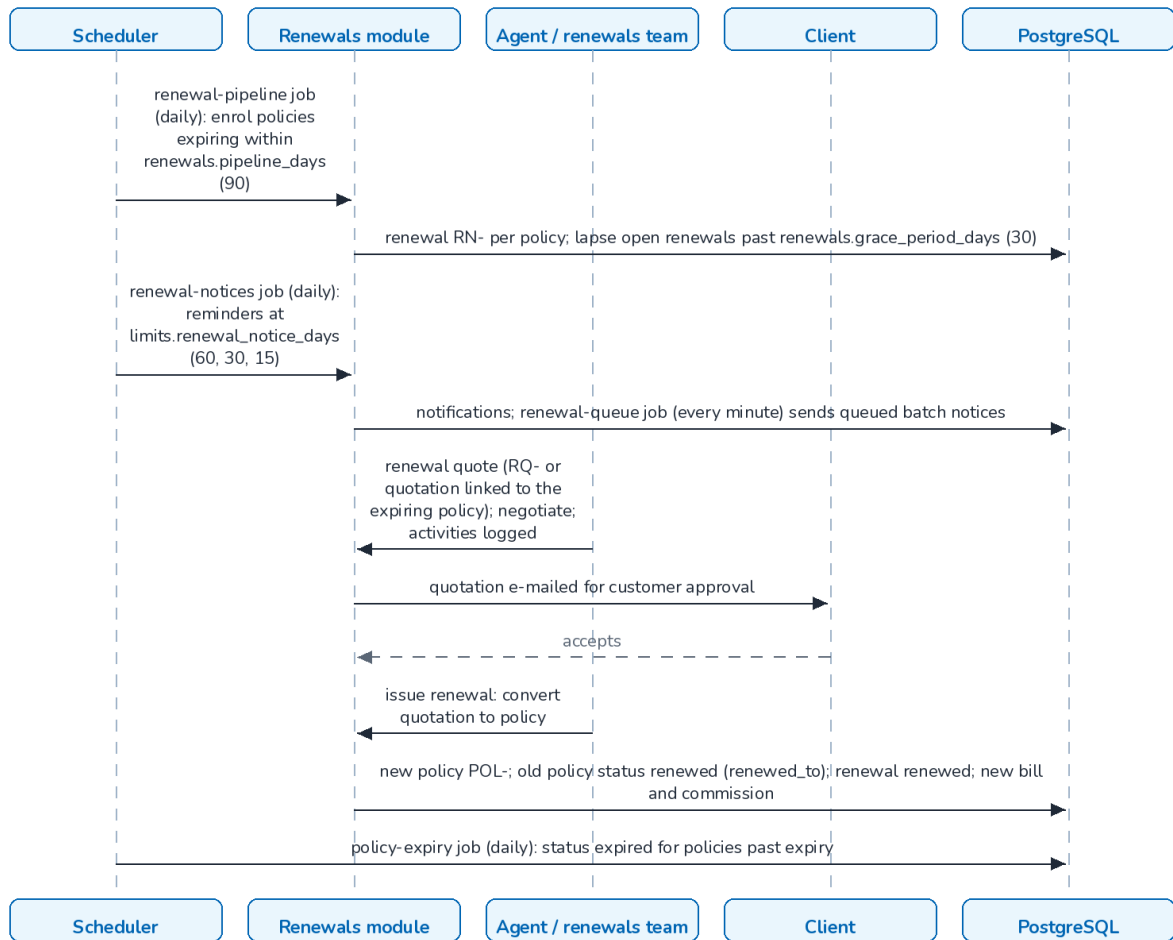


Figure 11. Renewal pipeline to renewed policy

8.6 Scheduled jobs

Job (scheduled_jobs.code)	Cron	Work
policy-expiry	15 0 * * *	Mark active / issued policies past their expiry date as expired
quote-expiry	30 0 * * *	Expire quotations older than <code>limits.quote_validity_days</code> (30)
daily-reports	0 5 * * *	Generate production register, collections summary and claims position; purge generated reports older than <code>reports.retention_days</code> (90)
renewal-pipeline	30 5 * * *	Enrol policies expiring within the pipeline window; lapse renewals past the grace period
renewal-notices	0 6 * * *	Renewal reminders at 60, 30 and 15 days (<code>limits.renewal_notice_days</code>)
receivable-ageing	0 7 * * *	Recompute age and ageing bucket of open bills
collection-reminders	0 8 * * *	E-mail and notify clients with premiums falling due
email-outbox	* / 5 * * * *	Deliver queued e-mails
renewal-queue	* * * * *	Send queued renewal batch notices, retry stale queue items

Report schedules created under Reports > Schedules add further jobs (`report-<id>`, handler `scheduledReport`).

Gap: node-cron is started without a time zone, so cron expressions are evaluated in the container's time zone. The Docker image and the local system run in UTC (the last runs of `renewal-notices` were at 06:00 UTC, 14:00 Manila time). The daily jobs therefore run 8 hours later than their cron suggests for Asia/Manila users. Set `TZ=Asia/Manila` on the API containers or pass `general.timezone` to `cron.schedule` (recommended fix).

9 Architecture decisions and constraints

Decision	Rationale and consequence
Modular monolith	One deployable API with module folders keeps transactions (policy, bill, journal, commission) in one database transaction; modules can be split later along the folder boundaries.
Stateless API	Sessions are tokens; state is in PostgreSQL and the file store, so instances can be added behind a load balancer (document 07). In-process caches (settings) and counters (rate limits) are per instance.
Migrations and seed on start	Every instance applies pending migrations in a transaction and the idempotent seed before it reports ready; simple to operate, but a failed migration blocks start-up (document 07, 09).
Configuration over code	273 settings in <code>app_settings</code> (tax rates, numbering prefixes, limits, templates, security policy), editable in Master > Configuration and System Settings.
No third-party document libraries	PDF, XLSX, CSV and ZIP writers are implemented in the code base, reducing dependencies and licence exposure; layout features are deliberately simple.
Local file store behind an S3-style API	<code>uploads/storage.js</code> keeps an S3-like key space on disk ("swap for S3 by replacing these functions"); production uses a shared volume (EFS) so all instances see the same files.

10 Related documents

This document is one of eleven that together form the BrokerVerse Solution Architecture. They are maintained and issued together (source and build: [docs/architecture/README.md](#)).

No.	Document	File
01	Solution Component Diagram and Application Architecture (this document)	01_BrokerVerse_Solution_Component_Diagram_Application_Architecture.pdf
02	Database Design	02_BrokerVerse_Database_Design.pdf
03	Database Inventory	03_BrokerVerse_Database_Inventory.pdf
04	Technology Stack	04_BrokerVerse_Technology_Stack.pdf
05	Shared Service Components	05_BrokerVerse_Shared_Service_Components.pdf
06	Capacity and Performance	06_BrokerVerse_Capacity_and_Performance.pdf
07	High Availability and Resiliency	07_BrokerVerse_High_Availability_and_Resiliency.pdf
08	RTO and RPO	08_BrokerVerse_RTO_and_RPO.pdf
09	Backup and Recovery	09_BrokerVerse_Backup_and_Recovery.pdf
10	Data Archival, Housekeeping and Restoration	10_BrokerVerse_Data_Archival_Housekeeping_and_Restoration.pdf
11	Monitoring	11_BrokerVerse_Monitoring.pdf

11 Glossary

Terms and abbreviations used in this document.

Term	Meaning
Advisory lock	PostgreSQL application-level lock (pg_try_advisory_lock) used by the scheduler so a job runs on one API instance at a time.
ALB	Application Load Balancer (AWS): distributes HTTPS requests over the API containers and checks their health.
API	Application programming interface; here the BrokerVerse REST API under /api served by the Node.js backend.
Audit log / audit trail	Table audit_log: who changed which record, when, from which IP, with the before and after values.
AZ	Availability Zone: an isolated data-centre group inside an AWS region.
CloudFront	AWS content delivery network that serves the single-page application and can route /api to the backend.
CloudWatch	AWS monitoring service: logs, metrics, dashboards and alarms.
CORS	Cross-Origin Resource Sharing: browser rule set that the API restricts to the configured web origins (CORS_ORIGINS).
CRA / craco	Create React App (react-scripts) build tool chain, customised with craco (Create React App Configuration Override).
Cron	Time-based schedule expression (minute hour day month weekday) used by the scheduled jobs.
CSP	Content Security Policy: HTTP header restricting what a page may load or run.
Debit note (DN)	Commission debit note raised by the broker to an insurer for direct-billed policies.
Direct bill	Billing mode in which the insurer collects the premium and the broker bills the insurer for its commission.
ECR	Elastic Container Registry (AWS): stores the backend container images.
ECS / Fargate	Elastic Container Service (AWS) and its serverless compute mode for running the API containers.
EFS	Elastic File System (AWS): shared, multi-AZ network file system proposed for the upload directory.
HMAC	Keyed-hash message authentication code; signs file links and hashes one-time codes.
HS256	HMAC-SHA256 JSON Web Token signature algorithm, the only one the API accepts.
JWT	JSON Web Token: signed token carrying the user id, roles, permissions and token version.
KYC	Know your customer: identification data required before a policy is issued.
Maker-checker	Control in which a record created or submitted by one user must be approved by a different user.
Migration	Numbered SQL file in backend/src/db/migrations applied once, in order, on API start.
Multi-AZ	Deployment with a synchronous standby in a second Availability Zone and automatic failover (RDS).
nginx	Web server used in the web container to serve the SPA and forward /api to the API container.
PITR	Point-in-time recovery: restoring a database to any second inside the backup retention window.
RDS	Amazon Relational Database Service, the managed PostgreSQL target.
Receivable / bill	Premium bill (INV-) raised when a policy, endorsement or renewal is issued.
Record scope	Rule that users holding only scoped roles (default: agent) see only their own records (lib/scope.js).

Term	Meaning
Refresh token	Long-lived token (30 days) exchanged for a new access token; rotated on every use.
Remittance	Payment of collected premium, net of commission, from the broker to the insurer.
RPO	Recovery point objective: the maximum acceptable data loss, measured in time.
RTO	Recovery time objective: the maximum acceptable time to restore service.
S3	Amazon Simple Storage Service: object storage hosting the SPA build (and proposed archive store).
Seed	Idempotent data load run on every API start: reference data always, sample data only when SEED_SAMPLE_DATA is on.
SMTP	Simple Mail Transfer Protocol; the API sends e-mail through the server in SMTP_URL.
SPA	Single-page application: the React front end served as static files.
TLS	Transport Layer Security (HTTPS).
Token version	users.token_version, carried in every access token; raising it ends all sessions of the user.
TOTP / 2FA	Time-based one-time password (RFC 6238) used for two-factor authentication.
UAT	User acceptance testing environment.
WAF	Web Application Firewall.
zod	TypeScript-first schema validation library used to validate request bodies and parameters.